Replacing AWS S3 with Cloudflare R2: Cost Analysis and Feasibility Study

Ekaterina Braun
Technical University of Munich
Advisor: Maximilian Kuschewski

München, April 2026

The real cost of object storage

- Cloud storage bills have three parts:

- **Storage:** *~$0.023/GB/month*
- **Operations:** *$0.4 per 1,000,000 GETs*
- **Egress:** *$0.09/GB* to the public internet

- For egress-heavy workloads (media delivery, ML data, backups), egress dominates the total bill

A radical alternative: Cloudflare R2

- **S3 Compatible API**
- **Durability SLO:** *99.999999999%* (same as S3)

- **Zero egress fees**
- **Storage:** *$0.015/GB/month* (35% cheaper than S3 Standard)
- **Operations:** *$0.36 per 1,000,000* (10% cheaper than S3 Standard)

- Free tier: 10 GB storage + 10M GET/month

But is it actually viable?

Zero egress
Cost savings
Drop-in API

Unknown at high bandwidth
Untested at high concurrency
Latency characteristics unclear

What’s set out to answer

Throughput

→ Can R2 saturate high-bandwidth links?

Latency

→ What happens under high concurrency?

Cost

→ Is migration economically worth it?

Scope of this study

1. Single-client benchmarking
2. EC2: 10–200 Gbps
3. High concurrency
4. Large chunks (50–100 MB)
5. Random ranges from one object

EC2 Client

Single instance
High throughput

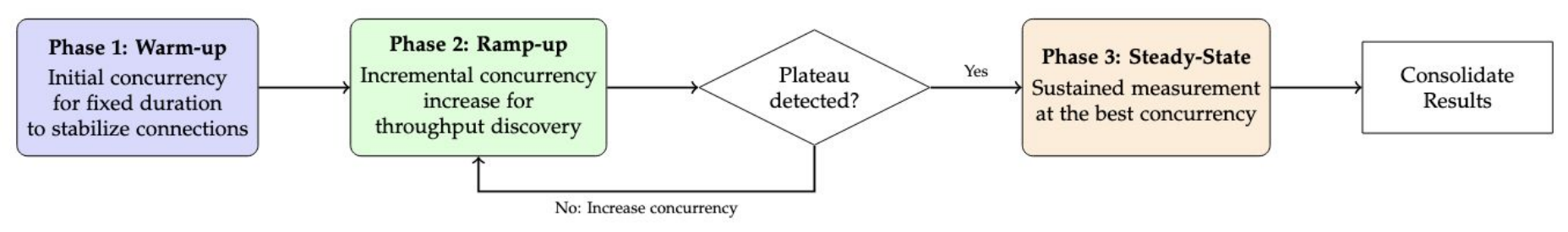
Internet

Multiflow TCP
High concurrency

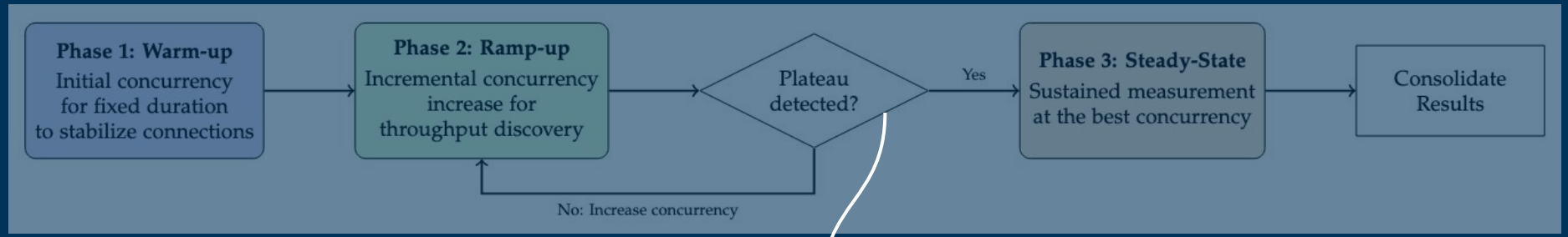
Cloudflare R2

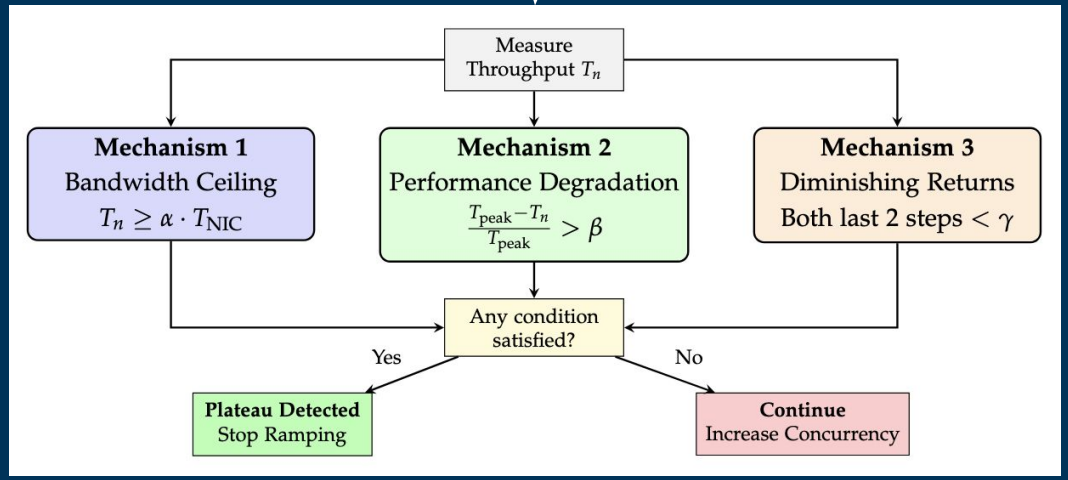
Object storage
One object, range GETs

Benchmarking methodology



Plateau detection

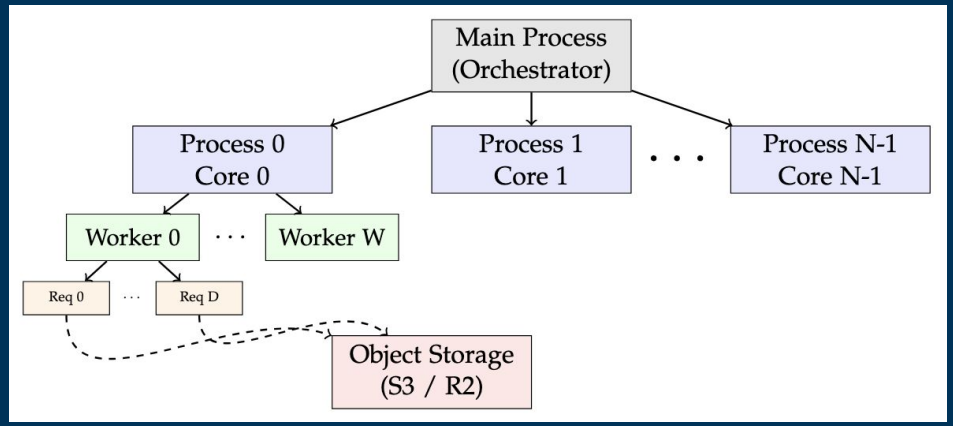




Custom benchmarking client

Python's GIL prevents naive multi-core parallelism → custom architecture

- One process per vCPU
- Async workers per process
- HTTP pipelining per worker



- Persists all requests to Parquet for offline analysis

R2 delivers — no throttling detected

Instance	NIC	vCPUs	Peak	Achieved	Needed	NIC Util.	Bottleneck
r5.xlarge	10 Gbps	4	6.7 Gbps	1.53	2.50	67%	CPU
r8gd.4xlarge	**15 Gbps**	16	**14.3 Gbps**	**0.81**	**0.94**	**95%** ✅	**NIC**
c5n.9xlarge	50 Gbps	36	41.0 Gbps	1.10	1.39	82%	CPU
c6in.16xlarge	100 Gbps	64	78.4 Gbps	1.15	1.56	78%	CPU
hpc7g.16xlarge	200 Gbps EFA	64	48.0 Gbps	0.70	3.13	24% ⚠️	EFA unsuitable
c6in.32xlarge	**200 Gbps**	128	**114 Gbps** ✅	0.81	1.56	57%	CPU

- AWS documents ~*100 Gbps* for S3 from a single EC2 instance.

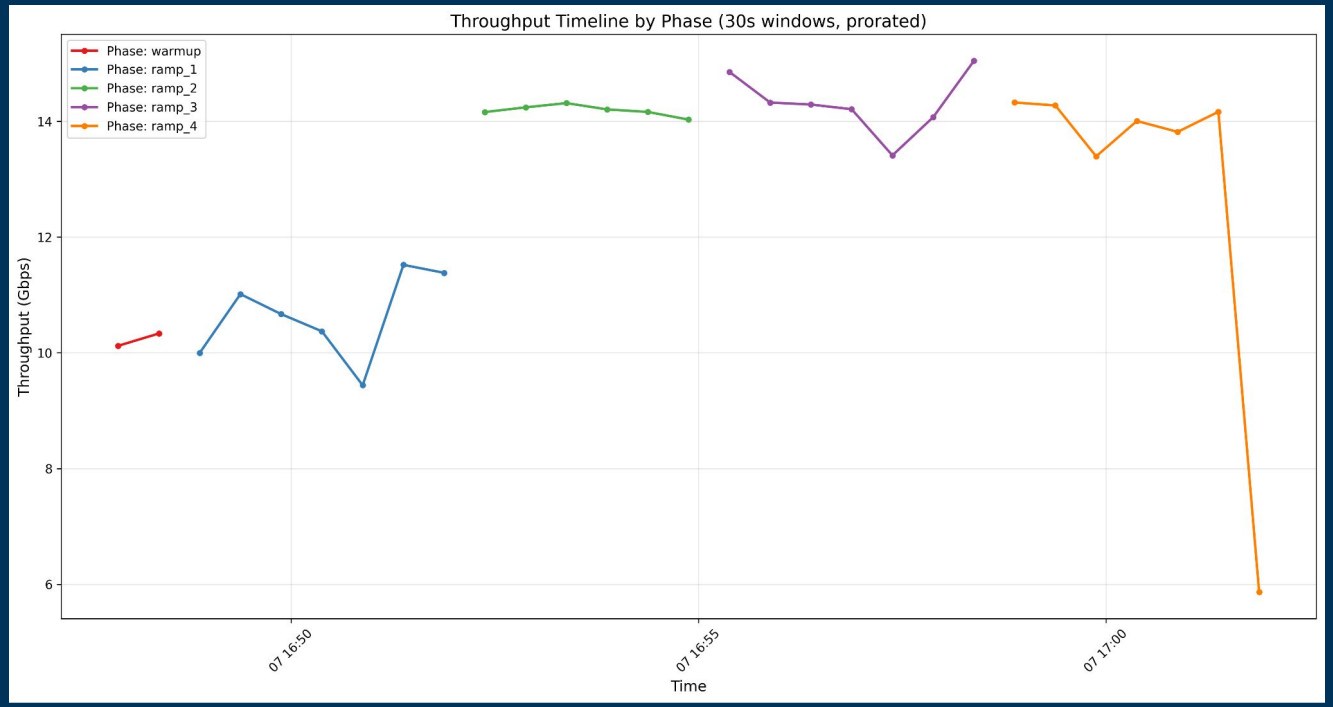
The client is the bottleneck. Not R2

- **CPU utilization:** 90–99% across all cores at peak

- Per-core throughput ceiling that falls when you add more cores:
  - *4 vCPUs → 1.5 Gbps/core*
  - *36–64 vCPUs → ~1.1 Gbps/core*
  - *128 vCPUs → ~0.8 Gbps/core*

- **At 16 vCPUs:** per-core need ~ actual for 15 Gbps. NIC fills first.
- **At 128 vCPUs:** per-core need > actual for 200 Gbps. CPU fills first.
  (114 Gbps is a lower bound)

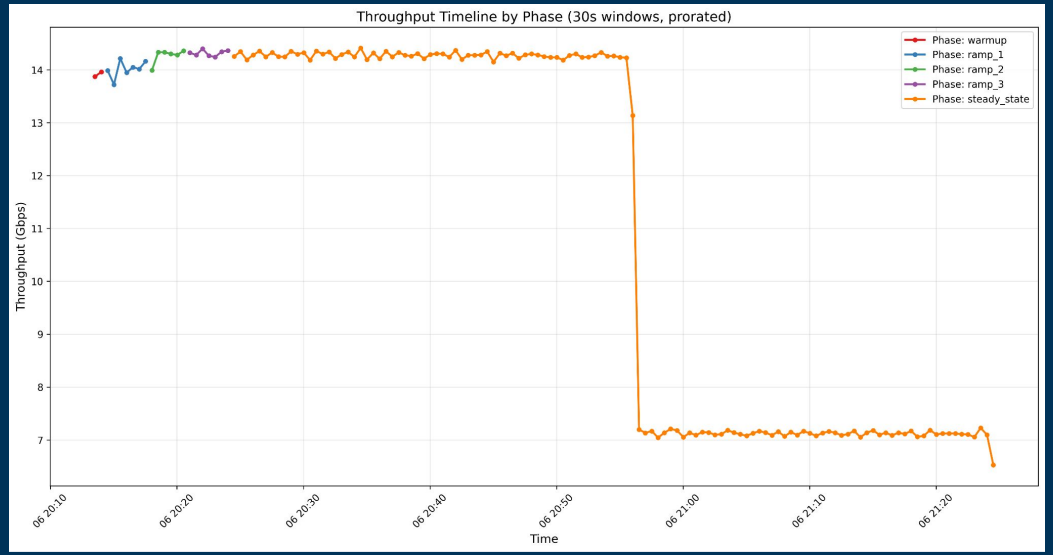
Client keeps up, R2 link saturated



r8gd.4xlarge (15 Gbps, 16 vCPUs)
clean ramp → stable plateau at 14.3 Gbps. 95% NIC utilization

Short benchmarks lie. The bandwidth cliff

- AWS **"up to N Gbps"** instances have a burst credit model
- Elevated bandwidth for a while, then drop hard
- Longer *1.2-hour* steady-state experiments to catch this



r8gd.4xlarge

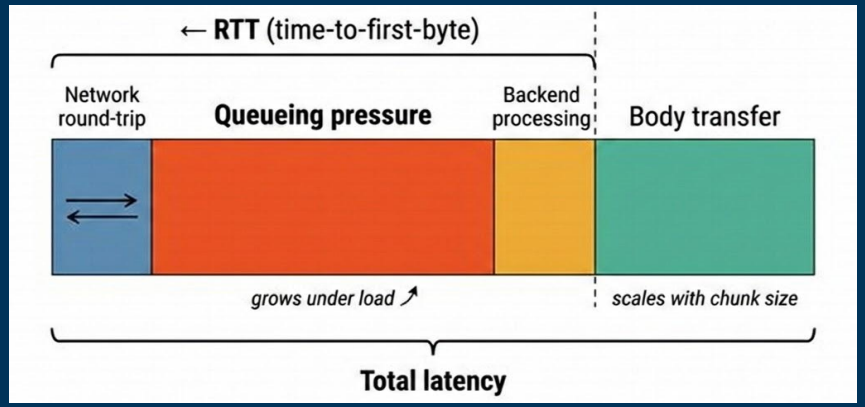
Two latency metrics

RTT (time-to-first-byte):

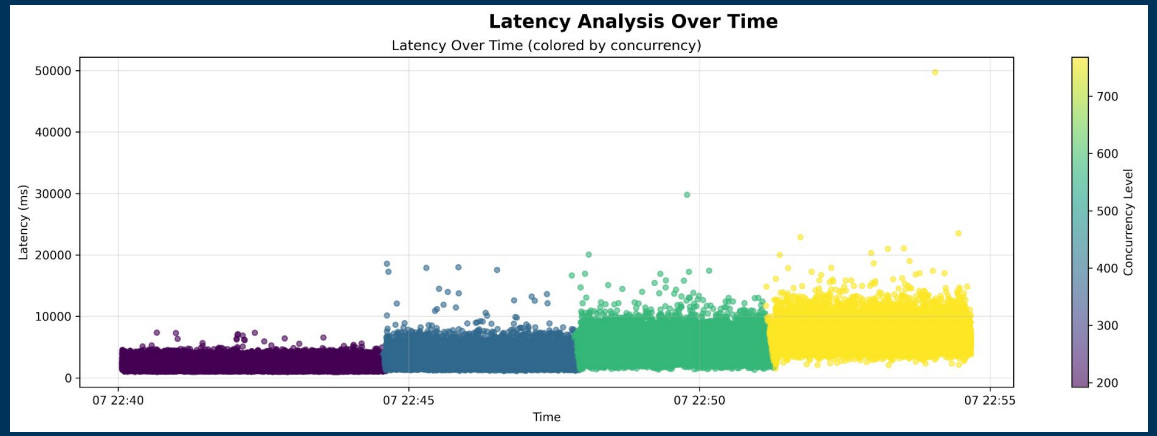
- Time from issuing the GET request to receiving the **first response byte**
- Independent of chunk size

Total latency:

- RTT + body transfer time
- Scales with chunk size
- Useful for **end-to-end** pipeline planning



Latency increases with concurrency



Clear staircase: median latency rises at each concurrency step; 100% success

State	Concurrency	RTT	Median latency
No load (warm-up)	C192	**~120 ms**	**~1.6 s**
Under load (peak)	C768	~4,323 ms	~6.8 s

Little's Law: 4× more concurrency → 4.3× higher latency — roughly proportional

At saturation, 62% time is waiting

Component	Time	Share
RTT (pre-transfer wait)	5,925 ms	**62%**
Body transfer	3,621 ms	38%
Total request time	**9,546 ms**	100%

c6in.32xlarge, 100 MB config, peak concurrency

- RTT elevated because requests queue up faster than R2 can respond
- NOT because the download itself is slow
- **Implication:** reducing queue depth matters

Request granularity matters

- Total in-flight data held constant:
  - *100 MB × pipeline=3 ≈ 50 MB × pipeline=6*

Config	Throughput	Avg Latency	RTT at peak concurrency
100 MB / p=3	109.3 Gbps	4,209 ms	**5,925 ms**
50 MB / p=6	110.7 Gbps	1,875 ms	**251 ms**
Change	**≈ same**	**−55%**	**−96%**

c6in.32xlarge *(same results with r8gd.4xlarge)*

Takeaway: Chunk size is the most impactful latency lever — without touching throughput.

Geography matters

Metric	Stockholm (eu-north-1)	Frankfurt (eu-central-1)
Avg throughput	**103 Gbps**	**89 Gbps** (–14%)
Concurrency at peak	C768	C1536 (2×)
RTT — median	148 ms	142 ms
RTT — p95	233 ms	**426 ms** (+83%)
Total latency — median	1.7 s	**2.8 s** (+66%)
Total latency — p95	3.3 s	**5.8 s** (+78%)

c6in.32xlarge: 128 vCPUs · 200 Gbps NIC · 50 MB chunks · pipeline=6

Cost impact

Cost comparison for different workloads:

Scale	Storage	Egress/mo	S3/month	R2/month	Savings	Migration cost	Break-even
Small	10 TB	1 TB	$320	$150	**53%**	~$920	~5 months
Medium	50 TB	10 TB	$2,050	$750	**63%**	~$4,400	~3.5 months
Large	100 TB	50 TB	$6,600	$1,500	**77%**	~$8,000	~1.6 months

Real-world example:

🏢 **VSCO** — migrated 2 PB of user images to R2.
Result: *$400K/year* in cloud savings.

Key takeaways

Throughput

R2 delivered *95%* NIC utilization at *15 Gbps*, *100%* success rate. At higher bandwidths, Python client was the bottleneck. *114 Gbps* is a lower bound on R2's capacity.

Latency

At no load: RTT *~120 ms*. Under peak load: *4×* concurrency → *4.3×* latency (predictable, stable). RTT = *62%* of request time at saturation. *50 MB* chunks reduce RTT by *96%* at equal throughput.

Cost

Huge monthly savings. Break-even in a couple of months. For large-object, egress-heavy workloads: the case is strong.

Future Work

1. Compiled-language client
2. More regions, instance types, etc for the same workload (paper ready)
3. Network-level diagnostics
4. S3 baseline with the current framework
5. Multi-object and small-object workloads
6. Write throughput

Python client (addressable by rewriting):

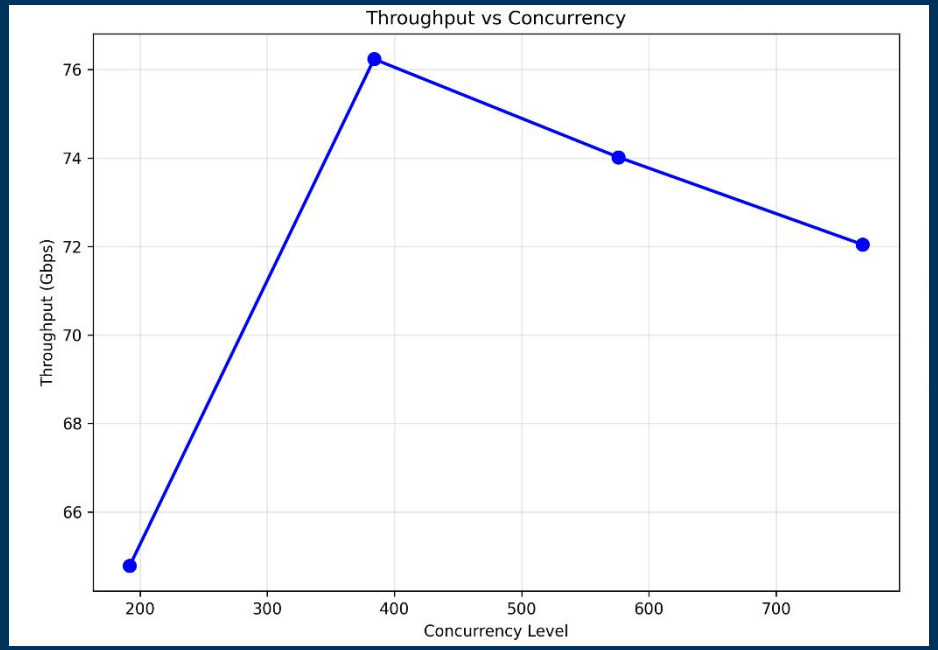
1. CPU at *90–99%* at peak; NIC has headroom
2. Per-core ceiling: *1.5 Gbps (4 vCPUs) → 0.8 Gbps (128 vCPUs)*
3. *2×* cores → only *1.4×* throughput
4. *114 Gbps* is a client ceiling, not R2's

R2 Storage (fundamental properties):

1. No throttling detected; *100%* success across all tests
2. NIC saturation confirmed: *95%* at *15 Gbps*
3. At no load: RTT *~120 ms* — comparable to S3
4. Latency grows proportionally with concurrency (stable, predictable)
5. At saturation: *62%* of request time is pre-transfer queueing
6. Chunk size controls queue depth — *96%* RTT reduction at same throughput

More concurrency, same throughput

CPU ceiling confirmed



c6in.16xlarge (100 Gbps, 64 vCPUs)
throughput peaks at C384, plateaus — not R2, CPU ceiling

Literature overview

- **Skyrise — TU Berlin (BTW 2025)**
  Multi-client benchmark: S3 scales to *~250 GiB/s* aggregate; *P50=27ms* but tail latency exceeds *10s* under load.
  → Multi-client scale. Our focus: single-client, high-bandwidth.
- **Tigris (2025)**
  For *1 KB* objects: *S3 P50=22ms* vs *R2 P50=606ms*.
  → R2 slower for small objects. Our focus: *50–100 MB* objects.
- **Cloudflare (2023)**
  For *1 MB* objects, geographically distributed: R2 *P95 is 38%* lower than S3.
  → R2's anycast routing advantages at multi-region scale.
- **Su et al. (2017)**
  Queueing theory applied to cloud object storage: latency follows predictable patterns near capacity.
  → Theoretical framework for interpreting our latency results.

When R2 makes sense

✅ Strong case	⚠️ Think twice
Large objects (50–100 MB+)	Small objects (< 1 MB)
High-throughput batch workloads	Latency-sensitive interactive APIs
Egress-heavy (media, ML, backups)	Deep AWS feature dependencies